第5讲 保留卷核对与解析

2025 第一次阶段测验 · 完整解析 · 请在独立完成测验后阅读

本文件包含2025第一次阶段测验全部24题的答案与完整解析。2026-10-11按用户要求逐题扩写，并接入第5讲网页，可按题展开阅读。

每题先列结论与原有训练评分，再补题意、计算或代码追踪。用户本次明确要求展示全卷解析；是否已经实际阅读、是否独立作答，仍需另行记录。

题目来源：无答案版（来源资料未随仓库分发）；参考并审校带答案版（来源资料未随仓库分发）。下面的推导、训练评分及材料问题说明由助手补充，不是官方新发布的答案或勘误。页码均为PDF文件页序。

按第5讲正文的单位制统计，最后按各讲卷面总分换算。理由正确但存在轻微计算失误时可给该单位0.5；不能因为猜对最终结果就把错误的解题方法当作已经掌握。

原课程第2讲 题1至3 六单位

第1题 PDF第2页 四单位

1. $10110101_2=128+32+16+4+1=181$。本题未指定补码解释，按非负二进制数转换。1单位。
2. $0101\ 1100_2=0x5C$，每四位一组。1单位。
3. `10110101 & 01011100 = 00010100`，所以 **0x14**。1单位。
4. b非零，`!b`为0，`!!b`为1，所以 **0x01**。逻辑非不是逐位取反。1单位。

逐步看这四问在做什么。 a、b是题给的两个8位位串；第(1)问没有指定补码类型，因此按普通非负二进制数理解。

二进制位，从左到右	1	0	1	1	0	1	0	1
这一位的权值	128	64	32	16	8	4	2	1
计入总和	128	0	32	16	0	4	0	1

只有位值为1的权值需要相加，所以得到181。若另行把该位串指定为8位补码，才会读成181−256=−75；这不是本题第(1)问的口径。

b分成 `0101` 和 `1100`，分别为十六进制5和C，所以是0x5C。按位与则必须同一位置上下比较，两位均为1才保留1：

```
  a = 1011 0101
  b = 0101 1100
---------------- &
      0001 0100 = 0x14
```

最后一问的 ！是逻辑非：非零变0，零变1。因为b=92非零，第一次得到0，第二次得到1。要求用十六进制填写，所以写0x01；它并不是把b每一位翻转两次。

第2题 PDF第2页 一单位

大于。10U是unsigned int，−10按通常算术转换变成32位unsigned的$2^{32}-10$，比10大。先做类型转换再比较；不能直接按数学整数比较。

原题比较的是C表达式中的−10和10U，而非脱离类型的两个数学整数。

1. `10`的类型是int，取负后−10仍是int；后缀U使10U为unsigned int。
2. 同等级的int和unsigned int参与比较，int先转换成unsigned int。
3. 本课程unsigned int为32位，转换得到 `-10 mod 2^32 = 4294967286`。
4. 实际比较的是4294967286和10，因此前者大于后者。

看起来在比较	转换后实际比较
−10 与 10U	4294967286U 与 10U

这里没有先算一次溢出的有符号加法；负数转unsigned的取模是类型转换规则。

第3题 PDF第2页 一单位

−12的16位补码是0xFFF4，符号扩展为int后是 **0xFFFFFFF4**。在x86-64小端机器上，从低地址到高地址的四字节是：

```
最低地址 -> F4 FF FF FF -> 最高地址
```

按本套测验前统一的“首字节=最低地址字节”，答案为 **0xF4**。原参考答案写0xFF，只有把“首字节”理解为书写位模式的最高有效字节等口径时才可能成立，不能把它教成小端最低地址字节。本次训练以已经说明的口径给分。

先求位模式，再按内存地址排列，不能把这两步合并。

```
+12的16位表示: 0000 0000 0000 1100
逐位取反:      1111 1111 1111 0011
再加1:          1111 1111 1111 0100 = 0xFFF4
扩成32位:      1111 1111 1111 1111 1111 1111 1111 0100
```

short是有符号类型，扩成int要保持−12这个数值，因此左侧补符号位1，得到0xFFFFFFF4。

若iy的起始地址为B，小端布局为：

内存地址	B	B+1	B+2	B+3
字节	F4	FF	FF	FF

所以“内存中的第一个字节”是F4；“把0xFFFFFFF4写在纸上时最左边的字节”是FF。带答案版的0xFF与本课程默认小端下的最低地址字节不同，必须保留这个口径说明，不能暗改原参考答案，也不能让学生把FF误记成小端规则。

原课程第3讲 题4至6 十单位

第4题 PDF第2页 四单位

正无穷是s=0、exp全1、frac全0，所以编码 **0x7F800000**，1单位。

对0xA1100000，拆得s=1、exp=66、frac=0x100000：

- **s=1**，1单位。
- **M=1+0x100000/2^{23}=1.125**，1单位。
- **E=66−127=−61**，1单位。

注意exp=66，E=−61，两者不是同一个量。

(1) 正无穷。 binary32共有1位符号、8位阶码、23位小数。无穷使用“阶码全1、小数全0”，正号再令s=0：

```
0 | 11111111 | 00000000000000000000000
按4位重新分组：0111 1111 1000 0000 0000 0000 0000 0000
十六进制：       7    F    8    0    0    0    0    0
```

(2) 拆0xA1100000。 先每个十六进制位写4个二进制位，再从左切出1、8、23位：

```
1010 0001 0001 0000 0000 0000 0000 0000
1 | 01000010 | 00100000000000000000000
s |   exp    |          frac
```

- 符号位为1，所以s=1。
- exp按无符号读是64+2=66，既非全0也非全1，因此用规格化公式；Bias=127，E=66−127=−61。
- frac对应二进制小数0.001，值为1/8。加上隐藏的前导1，M=1.001_2=1+1/8=1.125。

题目只让填s、M、E。如果要恢复整个数值，它是 `-1.125 × 2^(-61)`。exp不是E，frac也不是M。

第5题 PDF第2页 两单位

- `101.111100 → 110.000`。舍弃100恰为中点，保留尾位是1，向偶数侧进位；111加1产生向整数部分的进位。
- `101.100100 → 101.100`。也是中点，但保留尾位已为0，不进位。

每个结果与舍入依据合计1单位。

“向偶数”只在恰好中点时决定方向，看的是保留部分的最后一位。

保留三位小数后，相邻刻度相差2^(−3)=1/8。先用竖线分出保留和舍弃部分：

```
101.111 | 100
101.100 | 100
```

两题舍弃部分都是100，表示恰好半个刻度，即1/16。因此都需要比较两个候选的末位。

原数	较小候选	较大候选	保留末位为0的一侧
101.111100	101.111	110.000	110.000
101.100100	101.100	101.101	101.100

第一题，保留部分101.111的末位是1，要加一个最小保留单位0.001；小数111连续进位，得到110.000。第二题，101.100末位本来就是0，保持不变。

也可把候选都乘8，去掉小数点：第一题候选为47和48，选48；第二题为44和45，选44。“偶数”指这个刻度编号的偶数，不是只检查原数的整数部分5。

第6题 PDF第2至3页 四单位

A 是。 32位int全部可被double精确表示。先转double没有损失，再转float，与直接从该整数按相同舍入模式转float一致。1单位。

B 是。 泛泛说“浮点加法不满足结合律”在本题会失分。dx、dy、dz来自32位整数，任意两个和、三个和的整数精度需求都远小于double的53位有效精度，因此两条路径的加法都精确。1单位。

C 否。 两个32位整数的乘积可能需要超过53位有效精度，中间乘法就会舍入，分组顺序可以影响结果。一个可核算反例是x=y=134217725，z=5，按IEEE binary64逐次运算：

```
(dx*dy)*dz = 90071988520878112
dx*(dy*dz) = 90071988520878128
```

这些x、y、z都在非负32位int范围内。正确判断与“中间乘积失去精度”的理由各0.5单位；不要求背这个大数反例。

D 否。 例如x=0、z=1，则dx/dx为NaN，dz/dz为1，比较为假。即使两边都是NaN，相等比较仍为假。正确判断和反例各0.5单位。

关键前提：dx、dy、dz并非任意double，它们都来自32位int的精确转换。 原卷还排除了80位扩展精度，因此按binary64逐次运算。下面沿用课程的最近偶数舍入。

A：为什么两条转换路线相同?

```
路线1: x ──────────────→ float（可能舍入）
路线2: x → double（精确）→ float（可能舍入）
```

double有53位有效二进制精度，足以容纳32位int。两条路线在最终转float前对应的是同一个精确整数，因此按相同舍入规则得到相同float。结论不是“int转float永远精确”，而是“两边即使有损失，也以同样方式舍入”。

B：为什么本题可以移动加法括号? 两个32位int之和、三个32位int之和的数量级都不到2^33，远小于double可以连续精确表示整数的范围。这些加法都没有丢失有效位，所以两种顺序都等于精确的x+y+z。这里先转换后做double加法，不是在int类型中先把x+y算完再转double。

C：把反例的舍入展开。 取x=y=134217725、z=5，这些值都符合题目的整数范围。设N=134217725：

1. 精确N^2=18014397704175625。它在[2^53,2^54)中，double刻度间距为2；邻居为18014397704175624和18014397704175626。恰好中点，按最低保留位取偶数，舍入到前者。
2. 左路线把舍入后的乘积乘5，得到精确中间值90071988520878120。这里double间距为16，邻居为90071988520878112和90071988520878128，仍是中点；偶数刻度一侧为前者。所以左式得到90071988520878112。
3. 右路线先算N×5=671088625，这一步精确；再乘N，精确值为90071988520878125。它离上面的较小邻居13，离较大邻居3，所以舍入到90071988520878128。

两侧最终相差16。问题来自中间舍入，不是double范围溢出。

D：为什么不能直接说任何数除自身都是1? 这个规则要求分子分母非零。题目没有排除0；令x=0、z=1，就出现0.0/0.0与1.0/1.0。课程IEEE模型下前者为NaN、后者为1，比较不相等。NaN表示“不是一个数”，与任何值（包括自身）的相等比较均为假。

原课程第4讲 题7至9 六单位

第7题 PDF第3页 两单位

rbx低32位是 **ebx**；rcx最低字节是 **cl**。每项1单位。

这些名字指同一个物理寄存器的不同宽度部分。

```
rbx: 完整64位
ebx: rbx的低32位
 bx: rbx的低16位
 bl: rbx的低8位
```

rcx也有对应层级rcx、ecx、cx、cl，所以最低一个字节就是cl。一个字节为8位；题目问的是名称，不需要写其中存放的数值。

第8题 PDF第3页 两单位

通用寄存器访问8次，数据内存访问4次，各1单位。每条mov涉及一个提供地址的寄存器以及一个读入或写出的数据寄存器，共两次寄存器访问；每条mov读或写一次内存。不要把地址计算本身再算成一次内存访问，也不要计取指。

每条指令都分别数“读/写哪个通用寄存器”和“访问几次数据内存”。

原指令	通用寄存器访问	数据内存访问
movq (%rdi), %rax	读rdi中的地址；写rax，共2次	读rdi指向的8字节，1次
movq (%rsi), %rdx	读rsi中的地址；写rdx，共2次	读rsi指向的8字节，1次
movq %rdx, (%rdi)	读rdx中的数据；读rdi中的地址，共2次	写rdi指向的8字节，1次
movq %rax, (%rsi)	读rax中的数据；读rsi中的地址，共2次	写rsi指向的8字节，1次
合计	8次	4次

这是按题目给定的体系结构层面读写次数计数，不是数实际电路访问或缓存事务。括号里的rdi/rsi也必须读取，CPU才知道去哪里访问内存。每次movq传8字节，仍算一次数据内存访问，不算8次。

这段代码先把两处旧值分别读进rax、rdx，再交换写回；若直接覆盖其中一处再去读旧值，原数据就会丢失。

第9题 PDF第3页 两单位

- `0x10+0xC000+0x0200×4 = 0xC810`。
- `0x30+0xC000×8 = 0x60030`。

每项1单位。第二种寻址没有基址寄存器，不能因此遗漏索引比例项。

统一公式：位移 + 基址寄存器内容 + 索引寄存器内容×比例。

第一项 `0x10(%rdx,%rcx,4)` 中，位移为0x10、基址为0xC000、索引为0x0200、比例为4：

```
0x0200 × 4 = 0x0800
0xC000 + 0x0800 = 0xC800
0xC800 + 0x10 = 0xC810
```

第二项 `0x30(,%rdx,8)` 的第一个逗号前为空，表示没有基址寄存器；此时rdx处在索引位置：

```
0xC000 × 8 = 0x60000
0x60000 + 0x30 = 0x60030
```

这里只计算有效地址。题目没有给内存内容，不能进一步推断这些地址里保存的值。

原课程第5讲 题10至14 十五单位

第10题 PDF第3页 两单位

```
subq %rbx, %rax
andq %rbx, %rax
```

每行1单位。cmp像减法但丢弃数值结果；test像按位与但丢弃数值结果。原答案中第一行末尾`%ra`是寄存器名截断，应为`%rax`。

AT&T语法源在前、目的在后，两条原指令都只留下条件码。

原指令	内部运算	为什么不改rax	对应写回指令
cmpq %rbx,%rax	rax−rbx	只用差设置标志，不保存差	subq %rbx,%rax
testq %rbx,%rax	rax & rbx	只用按位与结果设置标志，不保存结果	andq %rbx,%rax

例如rax=7、rbx=3：cmp检查7−3的结果，但rax仍是7；sub则把4写回rax。test检查7&3，但rax仍是7；and把3写回rax。此处讨论的是指令对寄存器和条件码的效果。

第11题 PDF第3至4页 三单位

1. **`setle %al`**。cmp实际比较有符号x与y，要求x<=y。1单位。
2. **把源的八位零扩展为32位，写入eax**。1单位。
3. **写eax同时清rax高32位**。这是x86-64对32位通用寄存器写入的规则，不是另执行了一条隐藏的汇编指令。1单位。

补全后逐行读：

```
cmpq   %rsi, %rdi  # 以x-y设置标志；x在rdi，y在rsi
setle  %al         # 有符号x<=y时写1，否则写0；只写al这1字节
movzbl %al, %eax   # 把al的0/1零扩展成32位int返回值
ret               # 返回调用处
```

long是有符号类型，所以用le（less or equal），不能换成无符号的be。该条件对应ZF=1或SF与OF不同。这里setle中的le是关系后缀，不是操作数宽度。

movzbl可拆成mov、z、b、l：传送、零扩展、从byte、到32位long宽度。假设al为1，扩展后eax为0x00000001；若al为0，eax为0。setle不会清除eax里其他旧位，所以后面的扩展有实际作用。

此外，x86-64写eax会自动使整个rax的高32位为0。movzbl负责把8位扩成32位，写32位寄存器的规则又清除高32位，这是两个相关但不同的层次。

第12题 PDF第4页 四单位

第一个片段缺少的两行分别为：

```
jg .L4   # x>y时走另一个分支
ret      # x<=y分支已经得到结果，必须在落入.L4前返回
```

每行1单位。不要只补条件跳转却漏掉第一个分支的ret，否则会继续执行另一个分支。也不要根据函数名absdiff擅自把表达式改成正的绝对差；题面的两个结果分别是x−y和y−x，在对应分支下非正（讨论无溢出的数学值）。

条件传送片段缺少 **`cmpq %rsi,%rdi`**，1单位。前面的sub改变过条件码，cmovg之前需要重新建立比较x与y的条件。

最后一问1单位：这种实现会先计算候选值，可能造成不必要的开销，也可能执行原本不应执行的危险操作或副作用。指出提前求值，并给出代价或风险即可。cmov带内存源时也不能当作“条件为假就绝不访问内存”的保证。

先忠实读C分支，再确定跳转。 原题的名字absdiff容易误导；给出的C实际是：

```
long absdiff(long x, long y) {
    long result;
    if (x <= y)
        result = x - y;
    else
        result = y - x;
    return result;
}
```

例如x=3、y=8，返回−5；交换为x=8、y=3，仍返回−5。不要擅自按函数名改成正的绝对值。以下C数值说明限定运算不发生有符号溢出。

分支版本完整代码：

```
absdiff:
    cmpq %rsi, %rdi  # 比较x与y
    jg .L4           # x>y时去else；其余情况继续向下
    movq %rdi, %rax  # rax=x
    subq %rsi, %rax  # rax=x-y
    ret              # 当前分支结束，不能继续落入else
.L4:
    movq %rsi, %rax  # rax=y
    subq %rdi, %rax  # rax=y-x
    ret
```

第一个空用jg，因为跳转目标是else分支，不是if分支。第二个空ret用于结束第一条路径；如果漏掉，就会继续执行.L4并覆盖刚算好的结果。

条件传送版本完整代码：

```
absdiff:
    movq %rdi, %rax  # 默认候选从x开始
    subq %rsi, %rax  # rax=x-y
    movq %rsi, %rdx
    subq %rdi, %rdx  # rdx=y-x，另一个候选
    cmpq %rsi, %rdi  # 重新比较x与y
    cmovg %rdx, %rax # 若x>y，用y-x覆盖默认候选
    ret
```

cmovg只负责在条件成立时复制候选，不会重新比较x、y。此前最后一条sub产生的是y−x的标志，不能直接拿它当x−y的比较结果，所以补cmpq。

为什么有时不适合这种实现？ 这段实现先准备两侧候选，再选择一个。若某个候选求值昂贵，会白算；若包含本应由条件保护的无效解引用、除零或副作用，提前求值可能改变行为。例如 `p ? *p : 0` 不能无条件先读取*p再选择。并不是所有if都应改成cmov。

第13题 PDF第4至5页 五单位

五个填空按顺序是 **0、1、0、mask<<n、x&mask**，各1单位。

追踪思路：rax累计结果，rdx是掩码，rcx保存移位量；每轮用r8算x&mask，再或入rax，最后让mask左移。循环继续条件是mask非零。

语言边界：原题的long掩码左移最终会触及符号位，不能把填好的C程序当作对任意n都无未定义行为的可移植代码。若要写定义清楚的教学实现，使用unsigned long掩码并约束移位量1—63；题目所给机器指令则按64位结果和x86移位量规则执行。n的有效移位量为0时，原机器循环可能不终止。这些不改变试卷要求恢复的五个操作。

按寄存器分工恢复C，不必一次看懂整段。

寄存器	本题角色	从哪里看出
rdi	x	每轮复制到r8作为输入
ecx/cl	n及移位量	开头将esi复制给ecx，salq使用cl
rdx	mask	初始化1，每轮左移，最后测试是否为0
rax	result	初始化0，每轮按位或累计，最后返回
r8	临时的x&mask	先复制x，再与mask按位与

```
movl %esi, %ecx  # 保存n，移位用cl
movl $1, %edx    # 写32位会清高32位，所以rdx=1
movl $0, %eax    # rax=0
jmp .L2          # 先检查条件，再执行循环体
.L3:
movq %rdi, %r8   # 临时值=x
andq %rdx, %r8   # 临时值=x&mask
orq %r8, %rax    # result |= 临时值
salq %cl, %rdx   # mask左移n位（机器实际使用移位量的低6位）
.L2:
testq %rdx,%rdx # mask&mask，用ZF判断mask是否为0
jne .L3          # 不为0则继续
rep; ret         # 返回；这里不表示重复执行循环体
```

按原题填空的完整C为：

```
long loop(long x, int n) {
    long result = 0;
    long mask;
    for (mask = 1; mask != 0; mask = mask << n) {
        result |= x & mask;
    }
    return result;
}
```

以机器位模式x=0xD6、n=2看前几轮：

mask	x&mask	累计result
0x01	0x00	0x00
0x04	0x04	0x04
0x10	0x10	0x14
0x40	0x40	0x54

之后mask继续每次左移2位，仍须继续循环到64位mask变0；x更高位为0，所以result不再变化。这个循环选取位0、2、4、6等位置的位，按位或累计。

原题恢复的是机器操作的C式表达；若要实际编写无符号位操作版本，应改用unsigned long并明确移位范围。不能把上面的signed long左移写法宣称为所有输入都合法。

第14题 PDF第5页 一单位

.L13。表索引从0起，rdi=2选第三个八字节表项；间接jmp读取该表项中的目标地址，而不是跳到表项本身的地址。

.quad每项存8字节的目标地址，rdi给出从0开始的表下标。

索引	表项地址	表项存放的跳转目标
0	.L1+0	.L11
1	.L1+8	.L12
2	.L1+16	.L13
3	.L1+24	.L14

`jmp *.L1(,%rdi,8)` 先计算表项地址 `.L1+2×8`，再从该8字节位置读取目标地址，最后转到.L13。星号表示这里是间接跳转。需要区分“表项所在位置.L1+16”和“表项里保存的代码地址.L13”。

本题rdi=2在表范围内，因此只需定位第三项；不能因为数字2就选第二项.L12。

原课程第6讲 题15至17 十七单位

第15题 PDF第6页 三单位

依次是 **rsp**、**加8**、**rbx**，每项1单位。本题目标不是rsp本身，因此可按“读旧栈顶、rsp增加、写目标寄存器”的题给步骤理解。

把地址和地址里的内容分开命名。 设执行前rsp=0x1000，地址0x1000处的8字节保存数值42：

时刻	rsp	rbx	内存读取
执行前	0x1000	原值	尚未读取
pop完成后	0x1008	42	从旧地址0x1000读取8字节

题干里的x就是rsp保存的地址，y是该地址处读出的内容。最终进入rbx的是y，不是地址x。pop让rsp增加8，表示这8字节不再属于当前有效栈顶；通常不会把原内存字节擦成0。

原题目的寄存器是rbx，因此按题目列出的读旧栈顶、加8、写rbx这三个步骤填即可，不需要讨论pop到rsp等特殊形式。

第16题 PDF第6页 十单位

第(1)问三个值是 **0x108**、**0x400649**、**0x110**，各1单位。call把下一条指令地址0x400649压入八字节栈槽，ret再取出该地址并回收这个槽。

第(2)问五项为 **x:rdi**、**y:rsi**、**a:rdi**、**b:rsi**、**s:rax**，各1单位。这里根据题给汇编的对应时刻辨认值所在位置，不是说C变量永远绑定同一个寄存器。

第(3)问1单位：**rbx属于被调用者保存寄存器**，遵守ABI的mult2须使其返回时恢复调用前的rbx。函数执行期间可以暂时改它；题目中的multstore自己也用push/pop保留了其调用者的rbx。

第(4)问1单位：按题目要求保留调用与参数顺序的常规实现，在call前交换rdi与rsi，可以用临时寄存器或合适的交换指令。不能用两条直接相互覆盖的mov：第一条会丢掉待保存的旧值。数学乘法结果相同，不代表按题设把参数对调时就不需要理解调用位置。

先看整个函数的任务：multstore收到x、y、dest，调用mult2算乘积，再把乘积写到dest指向的内存。

```
400640: push %rbx          # 保存multstore的调用者原来的rbx
400641: mov  %rdx,%rbx     # 第3参数dest在rdx；放入能跨call保留的rbx
400644: callq mult2        # 用rdi=x、rsi=y调用乘法函数
400649: mov  %rax,(%rbx)   # 将mult2返回的乘积写入*dest
40064c: pop  %rbx          # 恢复外层调用者的rbx
40064d: retq               # multstore返回

400650: mov  %rdi,%rax     # mult2：rax=a
400653: imul %rsi,%rax     # rax=a*b，即s
400657: retq               # 返回到400649，乘积仍在rax
```

(1) 栈指针和返回地址。 题目给的rsp=0x110是“准备执行call时”，前面的push已经执行，不能再扣一次push。

```
call前：rsp=0x110
call：   rsp=0x110-8=0x108
         将下一条指令地址0x400649保存到0x108～0x10F
         转去0x400650执行mult2
ret：    从0x108读取0x400649作为继续执行的位置
         rsp=0x108+8=0x110
```

返回地址是call之后的那条指令地址0x400649，不是call自己的0x400644，也不是被调用函数入口0x400650。本小问止于mult2的ret执行完，没有继续算multstore后面的pop与ret。

(2) 参数与返回值。 System V中前两个整数/指针参数用rdi、rsi。外层把它们叫x、y，内层把相同传入值叫a、b；变量名改变，不会自动换一组寄存器。s在计算结束时位于rax，作为整数返回值。

(3) 为什么dest放rbx? rdx是调用者保存寄存器，下一次调用可能改它。rbx是被调用者保存寄存器，mult2如果临时改了rbx，就有责任在返回前恢复。因此multstore能在call后继续用rbx中的dest。同时multstore也必须对自己的调用者负责，所以它自己用push/pop保存和恢复原rbx。“跨调用保持”是ABI责任，不是call自动备份了rbx。

(4) 交换参数怎样避免丢值? 在保留题给调用结构的实现中，可在call前加入：

```
movq %rdi, %rcx  # 暂存原x
movq %rsi, %rdi  # 第1参数变成y
movq %rcx, %rsi  # 第2参数变成x
```

只写 `movq %rsi,%rdi` 再写 `movq %rdi,%rsi` 会让两者都变成y。数学乘法虽可交换，题目仍要求理解按参数顺序传值；这里按题设非高级优化的常规调用模型作答。

第17题 PDF第6至7页 四单位

采用测验前统一的课堂示意模型，依次为 **v0寄存器、v1内存、v2寄存器、v3寄存器**，每项1单位。

核心原因是v1取了地址并交给另一个函数，需要一个能被解引用的对象存储位置；其他几个值在示意实现中可以放寄存器。仅凭原C源程序无法唯一断言实际编译器一定如此分配，优化器可能传播常量、内联或消除变量。若学生主动指出这种不唯一性，并能解释题目所要检查的“取地址”规则，不应判成没有掌握。

先修正题面才能讨论变量位置。 无答案版bit_not1写了未定义的p，带答案版已注明应为a；bit_not2中缺少分号也补上：

```
long bit_not1(long a) {
    long x = ~a;
    return x;
}
long bit_not2(long *p) {
    long x = *p;
    long y = ~x;
    return y;
}
long call_bitnot() {
    long v0 = 1010;
    long v1 = 1010;
    long v2 = bit_not1(v0);
    long v3 = bit_not2(&v1);
    return v2 + v3;
}
```

变量	题目示意答案	为什么
v0	寄存器	只需把数值1010作为参数传给bit_not1，无须提供v0的地址
v1	内存	&v1要提供一个真实可寻址对象，bit_not2还会通过*p读取它
v2	寄存器	返回的整数可在寄存器中保存，无须提供对象地址
v3	寄存器	同样只需数值，用于最后的加法

通用寄存器不是可以用普通C指针取地址的内存格子，因此在本题保留调用的示意模型里，v1需要可寻址存储，通常放在栈上。把v2说成“可放寄存器”也不等于它必须一直留在rax：计算v3的调用会使用rax返回另一个值，v2若仍需要，可以转存到受保护的寄存器等位置。

实际编译器还会考虑寄存器压力、优化级别、内联和常量传播，因此题面并不能唯一决定真实编译后的分配。这一限定与参考答案的课堂示意意图应同时保留。

原课程第7讲 题18至22 十单位

第18题 PDF第7页 两单位

完整两行：

```
shlq $4, %rdi
movl arr_data(%rdi,%rsi,4), %eax
```

每行1单位。每行四个int占16字节，先让rdi变成行字节偏移16x，再加4y。第二行多个空合起来只算一单位。

先算地址，再对照指令。 `arr_data[4][4]` 每行有4个int，每个int占4字节，所以一行占16字节。

```
arr_data[x][y]的地址
= arr_data起点 + x×一行字节数 + y×一个元素字节数
= arr_data起点 + 16x + 4y
```

题给代码按合法下标x、y分析，入口rdi、rsi分别提供x、y供题中地址计算。左移4位就是乘16：

```
get_digit:
    shlq $4, %rdi                    # rdi从x变成16x
    movl arr_data(%rdi,%rsi,4),%eax # 地址=数组起点+16x+4y；读取4字节
    ret                              # int结果在eax
```

因此第一行目的为rdi；第二行的索引是rsi、比例为4、目的为eax。movl取一个int，而非整个8字节。不要把“左移4位”误算成乘4；乘4是左移2位。

第19题 PDF第7至8页 两单位

原片段缺少的两行是：

```
imulq %rdi, %rax
addq %rcx, %rax
```

每行1单位。rax先取得i，再乘n、加j；题面已经给出的`movl (%rsi,%rax,4),%eax`最后以四字节为比例、以a为基址读取int，不是待填行。n是运行时的列数，所以要保留乘法。

变化的是每行的长度：n在运行时才知道，不能直接写死一行16字节。

参数	入口寄存器	意义
n	rdi	每行有n个int
a	rsi	数组起始地址，参数调整为指向一行的指针
i	rdx	行号
j	rcx	列号

目标元素前面有i行，每行n个元素，再加本行前面的j个元素，所以线性下标为$i*n+j$。*每个int占4字节：地址=a+4*(i*n+j)。

```
var_ele:
    movq  %rdx,%rax          # rax=i，原题已给
    imulq %rdi,%rax          # rax=i*n，第1个待填行
    addq  %rcx,%rax          # rax=i*n+j，第2个待填行
    movl  (%rsi,%rax,4),%eax # 读取地址a+4*(i*n+j)处的int，原题已给
    ret
```

注意最后寻址中的比例4已经负责乘元素大小，所以前面的imul只乘n，不能再多乘一次4。数组参数传递地址，不是把整个n×n数组复制进寄存器；n必须另行传入，函数才知道行跨度。

第20题 PDF第8页 一单位

按题给连续行优先机器布局，四项依次是 **1、7、5、3**，总和 **16**。1单位。

14对应第2行第4列；16对应第3行第1列；zip1[4]是第1行第4列。下标均从0计数。这里按课程物理布局模型解释；严格可移植C的指针算术还受所指数组对象边界约束，不应推广“把二维数组强转成int*后任意跨行下标都是标准保证”的结论。

题目按行优先的连续机器布局，把4×5数组从左到右排平。

行号	该行的线性下标	五个数
0	0、1、2、3、4	1、5、2、0、6

行号	该行的线性下标	五个数
1	5、6、7、8、9	1、5、2、1、3
2	10、11、12、13、14	1、5、2、1、7
3	15、16、17、18、19	1、5、2、2、1

逐项定位：

1. `pgh[2][3]`：第2行第3列，值1。
2. `linear_zip[14]`：14除以每行长度5，商2、余4，对应第2行第4列，值7。
3. `*(linear_zip+16)`：等价于本题布局下的线性第16项；16=3×5+1，对应第3行第1列，值5。
4. `zip1[4]`：zip1从pgh[1]开始，即第1行开头；往后第4项为3，不是从整个数组起点数第4项。

因此result=1+7+5+3=16。若按代码完整打印，输出为：

```
result: 16
```

第2、3项的手算沿用原题指定的连续布局模型，不因此推广为严格C中任意跨子数组的指针访问都合法。

第21题 PDF第8至9页 两单位

答案 **A=9、B=5**，各1单位。推导必须把对齐纳入，而不是把所有偏移都当成紧密排列。

1. q->t起于偏移8，前面是B个char；故`align(B,4)=8`，得到B∈{5,6,7,8}。
2. t占4字节，short数组从12开始；q->u起于32，故`align(12+2A,8)=32`，得到A∈{7,8,9,10}。
3. p->y起于184，前面的int矩阵占4AB字节；`align(4AB,8)=184`，得176<4AB<=184，即AB为45或46。
4. 在前两步的有限候选中，只有A=9、B=5满足。

先把三条汇编翻译成三个“成员起点”条件。 原定义是：

```
typedef struct {
    int x[A][B];
    long y;
} str1;
typedef struct {
    char array[B];
    int t;
    short s[A];
    long u;
} str2;
```

```
movslq 8(%rsi),%rax   # 从q+8取4字节有符号int并扩成long，因此t偏移8
addq 32(%rsi),%rax    # 加q+32处的8字节long，因此u偏移32
movq %rax,184(%rdi)   # 写入p+184处，因此y偏移184
```

第一条约束：B到底能多大？ array从偏移0开始，占B字节，下一成员int必须从4的倍数开始。要“向上对齐到8”，原结束位置必须大于4且不超过8。因此B为5、6、7、8。B=4会让t直接从4开始，不能选；B=8本身已对齐，可以选。

第二条约束：A到底能多大？ t从8开始占4字节，到12；short数组从12开始，占2A字节，结束位置为12+2A。接着long u要求8字节对齐。要向上对齐到32，结束位置须满足24<12+2A≤32，因此6<A≤10，A为7、8、9、10。

补充问答：Q034：align(12+2A,8)中的8代表什么，为什么不是固定补8字节？ 。

第三条约束：AB是多少？ str1的int矩阵从0开始，占4AB字节；long y起点184是对齐后的结果。前一个8的倍数是176，因此176<4AB≤184。除以4得44<AB≤46，所以整数乘积只能45或46。

45在候选范围内只能是9×5；46只能分解成1×46或2×23，均不满足候选。因此A=9、B=5。

最后代回验证，防止只凑出数字。

str2部分	偏移范围	说明
array[5]	0～4	5字节
填充	5～7	使int从8开始
t	8～11	4字节
s[9]	12～29	9×2=18字节
填充	30～31	使long从32开始
u	32～39	8字节

str1的矩阵占9×5×4=180字节，即0～179；180～183填充，y从184开始。三个汇编偏移全部吻合。

第22题 PDF第9页 三单位

b在rsi、c在xmm0、result在xmm0，各1单位。a和b都是指针，依次占rdi/rsi；c才是第一个浮点值参数。返回double使用xmm0。

指令中的整数转double与float转double都是数值转换，不能把它们当成仅改变解释方式的原位字节搬运。

类型决定走哪条参数通道，不能只看总参数排名。

```
double mix_compute(long *a, double *b, float c) {
    double result = *a + *b + c;
    return result;
}
```

a和b都是地址，依次使用第1、第2个整数/指针参数寄存器rdi、rsi。c虽然是整个参数表的第三项，却是第一个浮点值参数，所以用xmm0。返回double也用xmm0。

汇编	此时在做什么
movsd (%rsi),%xmm1	从b指向的位置取double，即*b
movq (%rdi),%rax	从a指向的位置取long，即*a
cvtsi2sd %rax,%xmm2	将long数值转换成double，放xmm2
addsd %xmm1,%xmm2	xmm2得到转换后的*a加*b
cvtss2sd %xmm0,%xmm0	把入口float c转换成double
addsd %xmm2,%xmm0	加上前面部分，得到result
ret	以xmm0中的double作为返回值

因此b所在寄存器是rsi，不是装着*b的xmm1；c与result都回答xmm0，但分别对应入口与返回前两个时刻，不表示同一时刻它能独立保存两个值。ss是标量单精度，sd是标量双精度。

原课程第8讲 题23至24 四单位

第23题 PDF第9至10页 一单位

题目明确改为大端，带答案版写出的两个数值是（严格的printf输出大小写见本题末尾）：

```
Ints 0-1 == [0xC0C1C2C3,0xC4C5C6C7]
```

整项1单位。不要把默认x86小端前提压过题目明确给出的大端条件。

先写每个地址里的字节，再按题目指定的大端组合。 原代码为：

```
union {
    unsigned char c[8];
    unsigned int i[2];
} dw;
for (j = 0; j < 8; j++)
    dw.c[j] = 0xC0 + j;
```

相对起点的偏移	0	1	2	3	4	5	6	7
c[j]写入内容	C0	C1	C2	C3	C4	C5	C6	C7
被哪个int覆盖	i[0]	i[0]	i[0]	i[0]	i[1]	i[1]	i[1]	i[1]

联合成员从同一起点开始；每个unsigned int占4字节，所以i[0]读前4字节、i[1]读后4字节。大端让低地址字节放在整数高位，因此分别是0xC0C1C2C3和0xC4C5C6C7。这里不会把八个字节数值相加，也不会像默认小端那样反向组合。

原答案数值与实际输出格式的区别。 带答案版用大写十六进制写上述两值，数值正确；原printf使用小写格式符%x，因此严格按程序打印的整行是：

```
Ints 0-1 == [0xc0c1c2c3,0xc4c5c6c7]
```

最后还有换行符。格式串没有逗号后的空格；0x前缀由格式串中写死的字符提供。考试只填两个数值时用大写字母表示同一个十六进制数不改变数值结论，但“完整输出”应按实际大小写写。

第24题 PDF第10页 三单位

每个待填行是一单位，原表已经给出的安全库函数行不计分。按课程历史语境、讨论“引入新机制需要什么支持”，参考表如下：

待填机制	改用户源代码	重新编译用户程序	引入相应CPU硬件能力
栈初始位置随机化	否	否	否
内存页的可执行权限检查	否	否	是，需要执行权限检查支持
编译器插入canary	否	是	否

硬件列需要解释：若题目意思是“在已经有NX能力的x86-64机器上启用不可执行栈，是否仍必须更换CPU”，答案应为否。原卷使用“是否需要更换新的CPU”但没有明确原机器能力；本次按测验前说明，接受上述历史能力口径，也接受写“现有机器支持时不需要更换”的准确回答。不得背成“每次开启NX就必须换CPU”。

三行的本质分别是系统布局、硬件权限支持与编译器插桩。每行机制归属和条件正确可得1单位；仅有一个格子写错而整体理解正确可得0.5。canary检查是比较保存值是否改变，不依靠大小关系。

先理解三列分别问谁需要动。

- 修改源代码：程序员是否要改变C代码中的调用或逻辑。
- 重新编译：即使C代码不变，是否也需要编译器重新生成加入防护的机器代码。
- CPU硬件：执行权限等机制是否需要相应硬件能力；原卷把它写成“更换新CPU”，存在前提省略。

原表的示范行是“使用fgets等安全库函数：是、是、否”。它表示修改输入处理代码并重新编译，不依赖更换CPU；仍然需要正确传入缓冲区容量并处理返回值。

栈初始位置随机化：否、否、否。 通常由系统在装载/建立进程时改变栈的位置。它不要求用户重写每个函数，也不要求编译器给每个函数加入检查代码；一般不引入新的CPU执行能力。这一行只讨论原题指定的栈初始位置随机化，不泛化成所有代码区域随机化都没有二进制要求。它提高地址预测难度，不修复越界写入。

内存页执行权限：原答案否、否、是。 系统给页设置权限，CPU在取指时检查能否执行。普通用户C代码不必为每次取指手写检查，也不必仅为这个页权限机制重新生成每个函数；但CPU必须具有对应权限检查能力。若旧机器没有，可能需要支持该能力的硬件；若现有CPU本来就支持NX，就不能说启用一次NX必须换CPU。写答案时应把“按原表历史前提”和“现代已有支持的机器”区分。

Canary：否、是、否。 程序员不必手写每个函数的检查；编译器在受保护函数开头插入保存保护值的指令，在返回前插入核对指令，因此要重新编译。比较、加载等可使用已有普通指令，不要求新CPU。检测到保护值不一致时通常终止程序；不能把损坏的数据自动修复。

这三行分别从系统地址布局、执行权限、函数中的检查代码介入。防护可组合使用，但都不能替代正确的边界检查。

得分后回看哪里

失分题号	复习位置
1—3、7—9	第1讲：位、类型、字节序、寄存器与寻址
4—6、10—12	第2讲：浮点、条件码与分支
13—17	第3讲：循环、跳表、调用与局部存储
18—24	第4讲：数据布局、浮点参数、联合和防护

最终报告分别保留：限时内训练分、限时后独立补做、获得提示后完成。它们回答不同的问题，不合并成一个看起来更高的成绩。

本次核验与更正记录

2026–10–11：逐题对照本地无答案版和带答案版，补全24题推导，并保留原训练单位制。所有题号、子问与待填代码行均按原卷出现顺序核对；两版分页略有差异，题目标题默认标无答案版PDF页序。第6题起于第2页、续第3页；第11题起于第3页、续第4页；第14题在无答案版第5页，带答案版答案续到第6页。

- 第3题：保留参考答案0xFF，明确小端最低地址字节应为0xF4，区分原题“首字节”的歧义。
- 第10题：原答案%ra补为%rax。
- 第12题：保留原C分支的减法方向及跳跃子编号，不因函数名absdiff擅自改题。
- 第17题：带答案版已注明~~p应为~~a；另补缺失分号，明确寄存器分配是题设示意模型。
- 第18题：补齐原题内存操作数缺失的寄存器前缀%。
- 第20题：保留课堂连续二维数组的机器布局口径，并说明严格C指针边界限制。

- 第23题：原参考数值正确，新增说明%x实际输出小写，完整输出相应更正。
- 第24题：照列参考表，并说明“需要硬件支持”不等于“已有NX支持仍必须更换CPU”。

以上推导与说明由助手整理，不冒充原试卷发布方勘误。